

LAPORAN PRAKTIKUM STRUKTUR DATA

PEKAN 9

Disusun Oleh:

Ihsanul Zaky EL-Muhammady

NIM:2511533001

Dosen Pengampu:

Dr.Wahyudi S.T , M.T

Asisten Laboratorium:

Zahran Azaria Anvaya



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG 2026

KATA PENGANTAR

Dengan penuh rasa syukur, saya mengucapkan terima kasih kepada Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga laporan praktikum "Implementasi Binary Tree dan Graph pada Java" ini dapat diselesaikan dengan baik.

Saya juga menyampaikan penghargaan kepada dosen pengampu Mata Kuliah Praktikum Struktur Data serta para asisten laboratorium yang telah memberikan arahan dan bimbingan selama proses praktikum berlangsung.

Laporan ini disusun sebagai dokumentasi kegiatan praktikum yang bertujuan untuk memahami konsep struktur data non-linear berupa Binary Tree dan Graph, serta implementasinya dalam bahasa pemrograman Java, mulai dari pembangunan pohon secara manual, penelusuran menggunakan Inorder, Preorder, dan Postorder, hingga pencarian pada graf menggunakan algoritma DFS dan BFS.

Saya menyadari laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan agar laporan ini dapat memberikan manfaat dan menambah pemahaman mengenai implementasi Binary Tree dan Graph pada Java.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	
DAFTAR ISI.....	
BAB I PENDAHULUAN	
1.1 Latar Belakang	
1.2 Tujuan	
1.3 Manfaat.....	
BAB II PEMBAHASAN	
2.1 Node_2511533001	
2.2 BTree_2511533001	
2.3 BTreeDriver_2511533001	
2.4 GraphTraversal_2511533001.....	
2.5 Tugas	
BAB III KESIMPULAN	
3.1 Kesimpulan	
3.2 Saran.....	
DAFTAR PUSTAKA.....	

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pengolahan data, tidak semua informasi cocok disimpan dalam struktur yang linear. Ada kalanya data memiliki hubungan hierarkis atau relasi antar elemen yang lebih kompleks, sehingga dibutuhkan struktur yang mampu merepresentasikan hubungan tersebut secara lebih alami. Di sinilah Binary Tree dan Graph menjadi relevan.

Binary Tree adalah struktur data berbentuk pohon di mana setiap simpul memiliki maksimal dua anak, yaitu kiri dan kanan. Struktur ini sangat berguna untuk merepresentasikan data yang bersifat hierarkis dan mendukung berbagai metode penelusuran seperti Inorder, Preorder, dan Postorder. Sementara itu, Graph adalah struktur yang lebih umum, di mana setiap simpul bisa terhubung ke simpul mana pun melalui edge, tanpa batasan arah maupun jumlah koneksi.

Melalui praktikum ini, kedua struktur tersebut dipelajari lebih dalam, termasuk cara membangun pohon, menelusuri simpul, serta melakukan traversal pada graph menggunakan dua pendekatan yang berbeda yaitu DFS dan BFS.

1.2 Tujuan

1. Memahami konsep dasar struktur data Binary Tree dan cara membangunnya secara manual.
2. Mengetahui perbedaan metode traversal Inorder, Preorder, dan Postorder pada Binary Tree.
3. Memahami konsep Graph dan cara merepresentasikannya menggunakan Adjacency List.
4. Mengetahui cara kerja DFS dan BFS dalam menelusuri simpul-simpul pada Graph.
5. Melatih kemampuan berpikir rekursif dalam menyelesaikan penelusuran struktur data pohon dan graf.

1.3 Manfaat

1. Meningkatkan pemahaman tentang struktur data non-linear yang sering digunakan dalam pengembangan sistem nyata.
2. Melatih kemampuan membangun dan menelusuri Binary Tree secara manual menggunakan Java.
3. Memberikan gambaran bagaimana Graph dan traversalnya diterapkan dalam kasus seperti peta, jaringan, maupun sistem rekomendasi.
4. Memperkuat pemahaman tentang rekursi sebagai teknik dasar dalam pemrograman berbasis struktur data.

BAB II

PEMBAHASAN

2.1 Node_2511533001

```
public class Node_2511533001 {
    int data_3001;
    Node_2511533001 left_3001;
    Node_2511533001 right_3001;

    public Node_2511533001(int data_3001) {
        this.data_3001 = data_3001;
        left_3001 = null;
        right_3001 = null;
    }

    public void setLeft_3001(Node_2511533001 node_3001) {
        if (left_3001 == null)
            left_3001 = node_3001;
    }

    public void setRight_3001(Node_2511533001 node_3001) {
        if (right_3001 == null)
            right_3001 = node_3001;
    }

    public Node_2511533001 getLeft_3001() {
        return left_3001;
    }

    public Node_2511533001 getRight_3001() {
        return right_3001;
    }

    public int getData_3001() {
        return data_3001;
    }

    public void setData_3001(int data_3001) {
        this.data_3001 = data_3001;
    }

    void printPreorder_3001(Node_2511533001 node_3001) {
        if (node_3001 == null)
            return;
        System.out.print(node_3001.data_3001 + " ");
        printPreorder_3001(node_3001.left_3001);
        printPreorder_3001(node_3001.right_3001);
    }

    void printPostorder_3001(Node_2511533001 node_3001) {
        if (node_3001 == null)
            return;
        printPostorder_3001(node_3001.left_3001);
        printPostorder_3001(node_3001.right_3001);
        System.out.print(node_3001.data_3001 + " ");
    }

    void printInorder_3001(Node_2511533001 node_3001) {
        if (node_3001 == null)
            return;
        printInorder_3001(node_3001.left_3001);
        System.out.print(node_3001.data_3001 + " ");
        printInorder_3001(node_3001.right_3001);
    }

    public String print_3001() {
        return this.print_3001("", true, "");
    }

    public String print_3001(String prefix_3001, boolean isTail_3001, String sb_3001) {
        if (right_3001 != null) {
            right_3001.print_3001(prefix_3001 + (isTail_3001 ? "| " : " "), false, sb_3001);
        }
        System.out.println(prefix_3001 + (isTail_3001 ? "\\--" : "/--") + data_3001);
        if (left_3001 != null) {
            left_3001.print_3001(prefix_3001 + (isTail_3001 ? " " : "| "), true, sb_3001);
        }
    }
}
```

Penjelasan:

Class ini adalah cetak biru dari setiap simpul yang ada di dalam Binary Tree. Setiap node menyimpan satu nilai integer di data_3001, serta dua referensi yaitu left_3001 dan right_3001 yang masing-masing menunjuk ke anak kiri dan anak kanan. Kalau belum punya anak, keduanya bernilai null.

Method setLeft_3001 dan setRight_3001 hanya mau mengisi pointer kalau posisi tersebut masih kosong, jadi tidak bisa menempa anak yang sudah ada. Selain itu, class ini juga membawa tiga method traversal yaitu printPreorder_3001, printPostorder_3001, dan printInorder_3001 yang bekerja secara rekursif. Perbedaannya hanya pada urutan kapan nilai node dicetak, apakah sebelum, di antara, atau setelah penelusuran ke anak kiri dan kanan.

Ada juga method print_3001 yang menampilkan struktur pohon secara visual di konsol dengan format garis bercabang.

2.2 BTree_2511533001

```
public class BTree_2511533001 {
    private Node_2511533001 root_3001;
    private Node_2511533001 currentNode_3001;

    public BTree_2511533001(){
        root_3001 = null;
    }

    public boolean search_3001(int data_3001){
        return search_3001(root_3001, data_3001);
    }

    private boolean search_3001(Node_2511533001 node_3001, int data_3001)
    {
        if (node_3001 == null)
            return false;
        if (node_3001.getData_3001() == data_3001)
            return true;
        if (node_3001.getLeft_3001() != null)
            if (search_3001(node_3001.getLeft_3001(), data_3001))
                return true;
        if (node_3001.getRight_3001() != null)
            if (search_3001(node_3001.getRight_3001(), data_3001))
                return true;
        return false;
    }

    public void printInorder_3001(){
        if (root_3001 != null) root_3001.printInorder_3001(root_3001);
    }

    public void printPreOrder_3001(){
        if (root_3001 != null) root_3001.printPreorder_3001(root_3001);
    }

    public void printPostOrder_3001(){
        if (root_3001 != null) root_3001.printPostorder_3001(root_3001);
    }

    public Node_2511533001 getRoot_3001(){
        return root_3001;
    }

    public boolean isEmpty_3001(){
        return root_3001 == null;
    }

    public int countNodes_3001(){
        return countNodes_3001(root_3001);
    }

    public int countNodes_3001(Node_2511533001 node_3001){
        if (node_3001 == null){
            return 0;
        }
        else {
            int count_3001 = 1;
            count_3001 += countNodes_3001(node_3001.getLeft_3001());
            count_3001 += countNodes_3001(node_3001.getRight_3001());
            return count_3001;
        }
    }

    public void print_3001(){
        if (root_3001 != null) root_3001.print_3001();
    }

    public Node_2511533001 getCurrentNode_3001(){
        return currentNode_3001;
    }

    public void print_3001(){
        if (root_3001 != null) root_3001.print_3001();
    }

    public Node_2511533001 getCurrentNode_3001(){
        return currentNode_3001;
    }

    public void setCurrent_3001(Node_2511533001 currentNode_3001) {
        this.currentNode_3001 = currentNode_3001;
    }

    public void setRoot_3001(Node_2511533001 root_3001) {
        this.root_3001 = root_3001;
    }
}
```

Penjelasan:

Class ini berperan sebagai manajer dari keseluruhan struktur Binary Tree. Ia menyimpan referensi ke root_3001 sebagai titik masuk utama pohon, serta currentNode_3001 untuk melacak simpul yang sedang aktif.

Semua operasi utama dipusatkan di sini. Method search_3001 menelusuri seluruh pohon secara rekursif untuk mencari nilai tertentu, memeriksa tiap simpul dari kiri ke kanan. Method countNodes_3001 menghitung jumlah total simpul dengan cara yang sama, menjumlahkan hasil rekursi dari cabang kiri dan kanan lalu ditambah satu untuk simpul saat ini. Tiga method traversal tersedia sebagai jembatan yang memanggil method serupa di class Node_2511533001, sehingga pemanggilannya cukup dari satu titik saja.

2.3 BTreeDriver_2511533001

```
public class BTreeDriver_2511533001 {
    public static void main(String[] args){
        //Membuat Pohon
        BTree_2511533001 tree_3001 = new BTree_2511533001();
        System.out.printf("Jumlah Simpul awal pohon: ");
        System.out.println(tree_3001.countNodes_3001());

        //menambahkan simpul data 1
        Node_2511533001 root_3001 = new Node_2511533001(1);
        tree_3001.setRoot_3001(root_3001);

        //menjadikan simpul 1 sebagai root
        System.out.println("Jumlah simpul jika hanya ada root");
        System.out.println(tree_3001.countNodes_3001());

        Node_2511533001 node2_3001 = new Node_2511533001(2);
        Node_2511533001 node3_3001 = new Node_2511533001(3);
        Node_2511533001 node4_3001 = new Node_2511533001(4);
        Node_2511533001 node5_3001 = new Node_2511533001(5);
        Node_2511533001 node6_3001 = new Node_2511533001(6);
        Node_2511533001 node7_3001 = new Node_2511533001(7);
        Node_2511533001 node8_3001 = new Node_2511533001(8);
        Node_2511533001 node9_3001 = new Node_2511533001(9);

        root_3001.setLeft_3001(node2_3001);
        node2_3001.setLeft_3001(node4_3001);
        node2_3001.setRight_3001(node5_3001);
        node4_3001.setRight_3001(node8_3001);
        root_3001.setRight_3001(node3_3001);
        node3_3001.setLeft_3001(node6_3001);
        node3_3001.setRight_3001(node7_3001);
        node6_3001.setLeft_3001(node9_3001);

        //Set root
        tree_3001.setCurrent_3001(tree_3001.getRoot_3001());

        System.out.println("Menampilkan simpul terakhir: ");
        System.out.println(tree_3001.getCurrentNode_3001().getData_3001());

        System.out.println("Jumlah simpul; setelah simpul 7 ditambahkan");
        System.out.println(tree_3001.countNodes_3001());

        System.out.println("InOrder: ");
        tree_3001.printInorder_3001();

        System.out.println("\nPreorder: ");
        tree_3001.printPreOrder_3001();

        System.out.println("\nPostorder:");
        tree_3001.printPostOrder_3001();

        System.out.println("\nMenampilkan simpul dalam bentuk pohon");
        tree_3001.print_3001();
    }
}
```

Penjelasan:

Class ini adalah tempat pohon benar-benar dibangun dan dijalankan. Dimulai dengan membuat objek BTree_2511533001 yang masih kosong, lalu simpul root dibuat dengan nilai 1 dan disetel sebagai akar pohon.

Setelah itu, delapan simpul lain dibuat dan disambungkan secara manual membentuk struktur pohon dengan dua level cabang. Simpul 2 dan 3 menjadi anak dari root, simpul 4 dan 5 anak dari 2, simpul 6 dan 7 anak dari 3, simpul 8 anak kanan dari 4, dan simpul 9 anak kiri dari 6. Setelah pohon selesai dibentuk, program mencetak jumlah simpul, hasil tiga jenis traversal, serta tampilan visual struktur pohon ke konsol.

Output:

```
Jumlah Simpul awal pohon: 0
Jumlah simpul jika hanya ada root
1
Menampilkan simpul terakhir:
1
Jumlah simpul; setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
Preorder:
1 2 4 8 5 3 6 9 7
Postorder:
8 4 5 2 9 6 7 3 1
Menampilkan simpul dalam bentuk pohon
|      /--7
|      /--3
|      |      \--6
|      |      \--9
|--1
|      /--5
|      \--2
|          /--8
|          \--4
```

2.4 GraphTraversal_2511533001

```
package pekan9_2511533001;
import java.util.*;

public class GraphTraversal_2511533001 {
    private Map<String, List<String>> graph_3001 = new HashMap<>();

    public void addEdge_3001(String node1_3001, String node2_3001) {
        graph_3001.putIfAbsent(node1_3001, new ArrayList<>());
        graph_3001.putIfAbsent(node2_3001, new ArrayList<>());
        graph_3001.get(node1_3001).add(node2_3001);
        graph_3001.get(node2_3001).add(node1_3001);
    }

    public void printGraph_3001() {
        System.out.println("Graf Awal (Adjacency List):");
        for (String node_3001 : graph_3001.keySet()) {
            System.out.print(node_3001 + " -> ");
            List<String> neighbors_3001 = graph_3001.get(node_3001);
            System.out.println(String.join(", ", neighbors_3001));
        }
        System.out.println();
    }

    public void dfs_3001(String start_3001) {
        Set<String> visited_3001 = new HashSet<>();
        System.out.println("Penelusuran DFS:");
        dfsHelper_3001(start_3001, visited_3001);
        System.out.println();
    }

    private void dfsHelper_3001(String current_3001, Set<String> visited_3001) {
        if (visited_3001.contains(current_3001)) return;
        visited_3001.add(current_3001);
        System.out.print(current_3001 + " ");
        for (String neighbor_3001 : graph_3001.getOrDefault(current_3001, new ArrayList<>())) {
            if (!visited_3001.contains(neighbor_3001)) {
                dfsHelper_3001(neighbor_3001, visited_3001);
            }
        }
        System.out.println();
    }
}
```



```

        dfsHelper_3001(neighbor_3001, visited_3001);
    }
}

public void bfs_3001(String start_3001) {
    Set<String> visited_3001 = new HashSet<>();
    Queue<String> queue_3001 = new LinkedList<>();
    queue_3001.add(start_3001);
    visited_3001.add(start_3001);
    System.out.println("Penelusuran BFS:");
    while (!queue_3001.isEmpty()) {
        String current_3001 = queue_3001.poll();
        System.out.print(current_3001 + " ");
        for (String neighbor_3001 : graph_3001.getDefault(current_3001, new ArrayList<>())) {
            if (!visited_3001.contains(neighbor_3001)) {
                queue_3001.add(neighbor_3001);
                visited_3001.add(neighbor_3001);
            }
        }
    }
    System.out.println();
}

public static void main(String[] args) {
    GraphTraversal_2511533001 graph_3001 = new GraphTraversal_2511533001();

    graph_3001.addEdge_3001("A", "B");
    graph_3001.addEdge_3001("A", "C");
    graph_3001.addEdge_3001("B", "D");
    graph_3001.addEdge_3001("B", "E");

    System.out.println("Graf Awal adalah: ");
    graph_3001.printGraph_3001();
    graph_3001.dfs_3001("A");
    graph_3001.bfs_3001("B");
}

```

Penjelasan:

Class ini mengimplementasikan struktur Graph menggunakan Adjacency List berbasis HashMap, di mana setiap key adalah nama simpul dan valuenya adalah daftar simpul yang terhubung langsung. Karena graph yang dibangun bersifat tidak berarah, setiap kali edge ditambahkan lewat addEdge_3001, koneksi dibuat di kedua arah sekaligus.

Method dfs_3001 menelusuri graph secara mendalam menggunakan rekursi. Dimulai dari simpul awal, ia terus masuk ke tetangga pertama yang belum dikunjungi sebelum berpindah ke tetangga berikutnya. Berbeda dengan itu, bfs_3001 menelusuri graph secara melebar menggunakan antrian. Semua tetangga dari simpul saat ini diproses lebih dulu sebelum melanjutkan ke level berikutnya. Di method main, graph dibangun dengan lima edge yang membentuk jaringan sederhana, lalu keduanya dijalankan dari titik awal yang berbeda untuk memperlihatkan perbedaan hasil penelusurannya.

Output:

```

Graf Awal adalah:
Graf Awal (Adjacency List):
A -> B,C
B -> A,D,E
C -> A
D -> B
E -> B

Penelusuran DFS:
A B D E C
Penelusuran BFS:
B A D E C

```

2.5 Tugas

1.PetaRumahSakit_2511533001

```
package pekan9_2511533001;

import javax.swing.BorderFactory;
import javax.swing.Box;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.SwingUtilities;
import java.awt.BasicStroke;
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.FontMetrics;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.GridLayout;
import java.awt.RenderingHints;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedHashMap;
import java.util.LinkedHashSet;
import java.util.LinkedList;
import java.util.List;
```

```

import java.util.Map;

import java.util.Queue;

import java.util.Set;


public class PetaRumahSakit_2511533001 extends JFrame {


    private static final long serialVersionUID = 1L;


    private Map<String, List<String>> graph_3001;
    private Map<String, int[]> posisi_3001;
    private Set<String> visitedNodes_3001;
    private List<String> pathNodes_3001;


    private JComboBox<String> comboAwal_3001;
    private JComboBox<String> comboTujuan_3001;
    private JTextArea hasilArea_3001;
    private GraphPanel_2511533001 graphPanel_3001;


    public PetaRumahSakit_2511533001() {
        graph_3001 = new LinkedHashMap<>();
        posisi_3001 = new LinkedHashMap<>();
        visitedNodes_3001 = new HashSet<>();
        pathNodes_3001 = new ArrayList<>();


        inisialisasiGraph_3001();


        setTitle("Pencarian Jalur Peta Rumah Sakit - 2511533001");
        setSize(900, 650);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout(5, 5));


        // Panel Atas

```

```
JPanel panelKontrol_3001 = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 8));  
panelKontrol_3001.setBackground(new Color(30, 60, 100));
```

```
String[] nodes_3001 = graph_3001.keySet().toArray(new String[0]);  
comboAwal_3001 = new JComboBox<>(nodes_3001);  
comboTujuan_3001 = new JComboBox<>(nodes_3001);  
comboTujuan_3001.setSelectedIndex(nodes_3001.length - 1);
```

```
JLabel lblAwal_3001 = new JLabel("Lokasi Awal :");  
JLabel lblTujuan_3001 = new JLabel("Lokasi Tujuan:");  
lblAwal_3001.setForeground(Color.WHITE);  
lblTujuan_3001.setForeground(Color.WHITE);
```

```
JButton btnBFS_3001 = new JButton("[ BFS ]");  
JButton btnDFS_3001 = new JButton("[ DFS ]");  
JButton btnReset_3001 = new JButton("[ RESET ]");
```

```
btnBFS_3001.setBackground(new Color(0, 180, 80));  
btnBFS_3001.setForeground(Color.WHITE);  
btnBFS_3001.setFont(new Font("Monospaced", Font.BOLD, 13));
```

```
btnDFS_3001.setBackground(new Color(200, 130, 0));  
btnDFS_3001.setForeground(Color.WHITE);  
btnDFS_3001.setFont(new Font("Monospaced", Font.BOLD, 13));
```

```
btnReset_3001.setBackground(new Color(180, 30, 30));  
btnReset_3001.setForeground(Color.WHITE);  
btnReset_3001.setFont(new Font("Monospaced", Font.BOLD, 13));
```

```
JPanel leftPanel_3001 = new JPanel(new GridLayout(2, 2, 5, 3));  
leftPanel_3001.setOpaque(false);  
leftPanel_3001.add(lblAwal_3001);  
leftPanel_3001.add(comboAwal_3001);
```

```

leftPanel_3001.add(lblTujuan_3001);

leftPanel_3001.add(comboTujuan_3001);


panelKontrol_3001.add(leftPanel_3001);
panelKontrol_3001.add(Box.createHorizontalStrut(30));
panelKontrol_3001.add(btnBFS_3001);
panelKontrol_3001.add(btnDFS_3001);
panelKontrol_3001.add(btnReset_3001);


add(panelKontrol_3001, BorderLayout.NORTH);


// Panel Graph
graphPanel_3001 = new GraphPanel_2511533001();
graphPanel_3001.setBorder(BorderFactory.createTitledBorder(" VISUALISASI GRAPH PETA RUMAH SAKIT "));
graphPanel_3001.setPreferredSize(new Dimension(900, 380));
add(graphPanel_3001, BorderLayout.CENTER);


// Panel Hasil
hasilArea_3001 = new JTextArea(6, 80);
hasilArea_3001.setEditable(false);
hasilArea_3001.setFont(new Font("Monospaced", Font.BOLD, 12));
hasilArea_3001.setBackground(new Color(20, 20, 20));
hasilArea_3001.setForeground(new Color(180, 255, 180));
hasilArea_3001.setText("Hasil Pencarian :\nJalur :\nNode Dikunjungi :\nJumlah Node Dikunjungi : 0");
JScrollPane scrollHasil_3001 = new JScrollPane(hasilArea_3001);
scrollHasil_3001.setBorder(BorderFactory.createTitledBorder(" HASIL PENCARIAN "));
add(scrollHasil_3001, BorderLayout.SOUTH);


// Event Listeners
btnBFS_3001.addActionListener(e -> {
    String awal_3001 = (String) comboAwal_3001.getSelectedItem();
    String tujuan_3001 = (String) comboTujuan_3001.getSelectedItem();

```

```

        BFS_3001(awal_3001, tujuan_3001);
    });

    btnDFS_3001.addActionListener(e -> {
        String awal_3001 = (String) comboAwal_3001.getSelectedItem();
        String tujuan_3001 = (String) comboTujuan_3001.getSelectedItem();
        DFS_3001(awal_3001, tujuan_3001);
    });

    btnReset_3001.addActionListener(e -> resetGraph_3001());
}

private void inisialisasiGraph_3001() {
    addEdge_3001("Pintu Masuk", "Lobi Utama");
    addEdge_3001("Pintu Masuk", "Parkiran");
    addEdge_3001("Lobi Utama", "Pendaftaran");
    addEdge_3001("Lobi Utama", "Apotek");
    addEdge_3001("Pendaftaran", "Poli Umum");
    addEdge_3001("Pendaftaran", "Poli Anak");
    addEdge_3001("Poli Umum", "Laboratorium");
    addEdge_3001("Poli Umum", "Radiologi");
    addEdge_3001("Poli Anak", "Radiologi");
    addEdge_3001("Laboratorium", "IGD");
    addEdge_3001("Radiologi", "IGD");
    addEdge_3001("IGD", "Ruang Operasi");
    addEdge_3001("IGD", "ICU");
    addEdge_3001("Ruang Operasi", "ICU");
    addEdge_3001("ICU", "Rawat Inap");
    addEdge_3001("Apotek", "Kasir");
    addEdge_3001("Kasir", "Pintu Keluar");

    posisi_3001.put("Pintu Masuk", new int[]{80, 180});
    posisi_3001.put("Parkiran", new int[]{80, 310});

```

```

posisi_3001.put("Lobi Utama", new int[]{220, 180});
posisi_3001.put("Pendaftaran", new int[]{360, 100});
posisi_3001.put("Apotek", new int[]{360, 260});
posisi_3001.put("Poli Umum", new int[]{490, 60});
posisi_3001.put("Poli Anak", new int[]{490, 160});
posisi_3001.put("Laboratorium", new int[]{620, 60});
posisi_3001.put("Radiologi", new int[]{620, 160});
posisi_3001.put("IGD", new int[]{730, 110});
posisi_3001.put("Ruang Operasi", new int[]{820, 60});
posisi_3001.put("ICU", new int[]{820, 160});
posisi_3001.put("Rawat Inap", new int[]{820, 260});
posisi_3001.put("Kasir", new int[]{490, 310});
posisi_3001.put("Pintu Keluar", new int[]{620, 310});
}

```

```

private void addEdge_3001(String a_3001, String b_3001) {
    graph_3001.putIfAbsent(a_3001, new ArrayList<>());
    graph_3001.putIfAbsent(b_3001, new ArrayList<>());
    graph_3001.get(a_3001).add(b_3001);
    graph_3001.get(b_3001).add(a_3001);
}

```

```

public void BFS_3001(String start_3001, String goal_3001) {
    Map<String, String> parent_3001 = new LinkedHashMap<>();
    Queue<String> queue_3001 = new LinkedList<>();
    Set<String> visited_3001 = new LinkedHashSet<>();

    queue_3001.add(start_3001);
    visited_3001.add(start_3001);
    parent_3001.put(start_3001, null);

    while (!queue_3001.isEmpty()) {
        String current_3001 = queue_3001.poll();
    }
}

```

```

        if (current_3001.equals(goal_3001)) break;

        List<String> neighbors_3001 = graph_3001.get(current_3001);
        if (neighbors_3001 == null) continue;
        for (String neighbor_3001 : neighbors_3001) {
            if (!visited_3001.contains(neighbor_3001)) {
                visited_3001.add(neighbor_3001);
                parent_3001.put(neighbor_3001, current_3001);
                queue_3001.add(neighbor_3001);
            }
        }
    }
}

List<String> path_3001 = bangunJalur_3001(parent_3001, start_3001, goal_3001);
displayPath_3001("BFS", new ArrayList<>(visited_3001), path_3001);
}

public void DFS_3001(String start_3001, String goal_3001) {
    Map<String, String> parent_3001 = new LinkedHashMap<>();
    Set<String> visited_3001 = new LinkedHashSet<>();
    parent_3001.put(start_3001, null);
    dfsHelper_3001(start_3001, goal_3001, visited_3001, parent_3001);

    List<String> path_3001 = bangunJalur_3001(parent_3001, start_3001, goal_3001);
    displayPath_3001("DFS", new ArrayList<>(visited_3001), path_3001);
}

private boolean dfsHelper_3001(String current_3001, String goal_3001,
                               Set<String> visited_3001, Map<String, String> parent_3001) {
    visited_3001.add(current_3001);
    if (current_3001.equals(goal_3001)) return true;
    List<String> neighbors_3001 = graph_3001.get(current_3001);
    if (neighbors_3001 == null) return false;
    for (String neighbor_3001 : neighbors_3001) {

```



```

        if (!visited_3001.contains(neighbor_3001)) {
            parent_3001.put(neighbor_3001, current_3001);
            if (dfsHelper_3001(neighbor_3001, goal_3001, visited_3001, parent_3001)) return true;
        }
    }
    return false;
}

```

```

private List<String> bangunJalur_3001(Map<String, String> parent_3001, String start_3001, String
goal_3001) {
    List<String> path_3001 = new ArrayList<>();
    if (!parent_3001.containsKey(goal_3001)) return path_3001;
    String curr_3001 = goal_3001;
    while (curr_3001 != null) {
        path_3001.add(0, curr_3001);
        curr_3001 = parent_3001.get(curr_3001);
    }
    return path_3001;
}

```

```

public void displayPath_3001(String metode_3001, List<String> visited_3001, List<String> path_3001) {
    visitedNodes_3001 = new HashSet<>(visited_3001);
    pathNodes_3001 = new ArrayList<>(path_3001);
    graphPanel_3001.repaint();

    StringBuilder sb_3001 = new StringBuilder();
    sb_3001.append("Hasil Pencarian : ").append(metode_3001).append("\n");
    sb_3001.append("Jalur      : ").append(path_3001.isEmpty() ? "Tidak ditemukan" : String.join(" -> ",
path_3001)).append("\n");
    sb_3001.append("Node Dikunjungi : ").append(String.join(", ", visited_3001)).append("\n");
    sb_3001.append("Jumlah Node Dikunjungi : ").append(visited_3001.size());
    hasilArea_3001.setText(sb_3001.toString());
}

```

```

public void displayGraph_3001(Graphics g_3001) {

    Graphics2D g2_3001 = (Graphics2D) g_3001;

    g2_3001.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

    // Gambar edge

    Set<String> drawnEdges_3001 = new HashSet<>();

    for (String node_3001 : graph_3001.keySet()) {

        int[] pos1_3001 = posisi_3001.get(node_3001);

        if (pos1_3001 == null) continue;

        List<String> neighbors_3001 = graph_3001.get(node_3001);

        if (neighbors_3001 == null) continue;

        for (String neighbor_3001 : neighbors_3001) {

            String edgeKey_3001 = node_3001.compareTo(neighbor_3001) < 0

                ? node_3001 + "-" + neighbor_3001

                : neighbor_3001 + "-" + node_3001;

            if (drawnEdges_3001.contains(edgeKey_3001)) continue;

            drawnEdges_3001.add(edgeKey_3001);

            int[] pos2_3001 = posisi_3001.get(neighbor_3001);

            if (pos2_3001 == null) continue;

            boolean isPath_3001 = false;

            if (pathNodes_3001.contains(node_3001) && pathNodes_3001.contains(neighbor_3001)) {

                int idx1_3001 = pathNodes_3001.indexOf(node_3001);

                int idx2_3001 = pathNodes_3001.indexOf(neighbor_3001);

                if (Math.abs(idx1_3001 - idx2_3001) == 1) isPath_3001 = true;

            }

            if (isPath_3001) {

                g2_3001.setColor(new Color(255, 220, 0));

                g2_3001.setStroke(new BasicStroke(3f));

            } else {

```

```

        g2_3001.setColor(new Color(120, 120, 120));

        g2_3001.setStroke(new BasicStroke(1.5f));

    }

    g2_3001.drawLine(pos1_3001[0], pos1_3001[1], pos2_3001[0], pos2_3001[1]);

}
}

```

// Gambar node

```

int r_3001 = 28;

for (String node_3001 : posisi_3001.keySet()) {
    int[] pos_3001 = posisi_3001.get(node_3001);
    Color fillColor_3001;

    if (pathNodes_3001.contains(node_3001)) {
        fillColor_3001 = new Color(255, 210, 0);
    } else if (visitedNodes_3001.contains(node_3001)) {
        fillColor_3001 = new Color(80, 160, 255);
    } else {
        fillColor_3001 = new Color(50, 50, 80);
    }

    g2_3001.setColor(fillColor_3001);
    g2_3001.fillOval(pos_3001[0] - r_3001, pos_3001[1] - r_3001, r_3001 * 2, r_3001 * 2);
    g2_3001.setColor(Color.WHITE);
    g2_3001.setStroke(new BasicStroke(1.5f));
    g2_3001.drawOval(pos_3001[0] - r_3001, pos_3001[1] - r_3001, r_3001 * 2, r_3001 * 2);

    g2_3001.setColor(Color.WHITE);
    g2_3001.setFont(new Font("Arial", Font.BOLD, 9));
    FontMetrics fm_3001 = g2_3001.getFontMetrics();
    String[] words_3001 = node_3001.split(" ");
    int lineH_3001 = fm_3001.getHeight();
    int startY_3001 = pos_3001[1] - (lineH_3001 * words_3001.length) / 2 + fm_3001.getAscent();
    for (String word_3001 : words_3001) {
        int w_3001 = fm_3001.stringWidth(word_3001);
    }
}

```

```

        g2_3001.drawString(word_3001, pos_3001[0] - w_3001 / 2, startY_3001);

        startY_3001 += lineH_3001;
    }
}

// Legend

g2_3001.setFont(new Font("Arial", Font.PLAIN, 11));

int lx_3001 = 15;
int ly_3001 = 20;

g2_3001.setColor(new Color(50, 50, 80));

g2_3001.fillOval(lx_3001, ly_3001, 14, 14);

g2_3001.setColor(Color.LIGHT_GRAY);

g2_3001.drawString("Belum dikunjungi", lx_3001 + 18, ly_3001 + 11);
ly_3001 += 20;

g2_3001.setColor(new Color(80, 160, 255));

g2_3001.fillOval(lx_3001, ly_3001, 14, 14);

g2_3001.setColor(Color.LIGHT_GRAY);

g2_3001.drawString("Dikunjungi", lx_3001 + 18, ly_3001 + 11);
ly_3001 += 20;

g2_3001.setColor(new Color(255, 210, 0));

g2_3001.fillOval(lx_3001, ly_3001, 14, 14);

g2_3001.setColor(Color.LIGHT_GRAY);

g2_3001.drawString("Jalur ditemukan", lx_3001 + 18, ly_3001 + 11);
}

public void resetGraph_3001() {
    visitedNodes_3001.clear();
    pathNodes_3001.clear();
    graphPanel_3001.repaint();
    hasilArea_3001.setText("Hasil Pencarian :\nJalur :\nNode Dikunjungi :\nJumlah Node Dikunjungi : 0");
}

class GraphPanel_2511533001 extends JPanel {

```

```

private static final long serialVersionUID = 1L;

public GraphPanel_2511533001() {
    setBackground(new Color(15, 15, 30));
}

@Override
protected void paintComponent(Graphics g_3001) {
    super.paintComponent(g_3001);
    displayGraph_3001(g_3001);
}
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(new Runnable() {
        public void run() {
            new PetaRumahSakit_2511533001().setVisible(true);
        }
    });
}
}

```

Penjelasan:

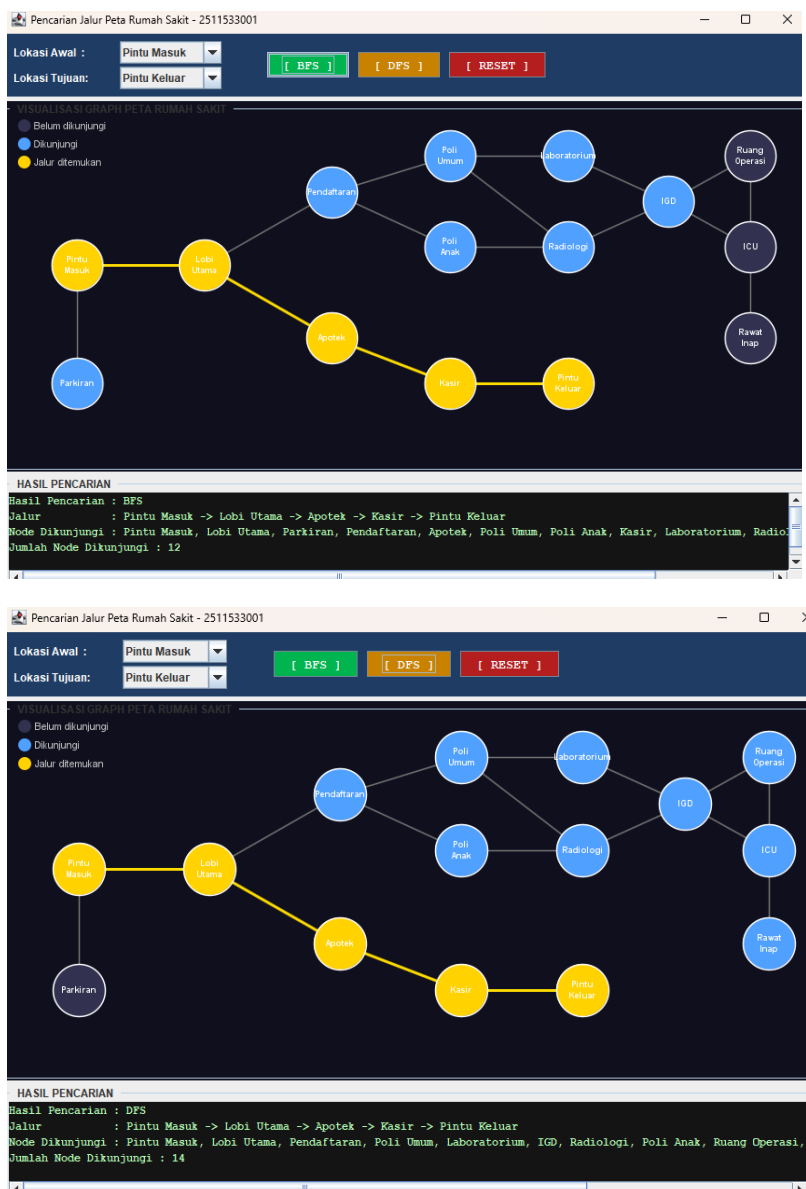
Class ini mengimplementasikan seluruh program dalam satu file, mencakup struktur graph, algoritma pencarian, dan tampilan GUI sekaligus. Studi kasus yang digunakan adalah peta area rumah sakit dengan 15 node dan 17 edge, merepresentasikan lokasi seperti Pintu Masuk, Pendaftaran, IGD, ICU, Radiologi, hingga Pintu Keluar. Graph disimpan menggunakan LinkedHashMap di mana setiap key adalah nama lokasi dan valuenya adalah daftar lokasi yang terhubung langsung. Karena graph bersifat tidak berarah, setiap kali edge ditambahkan lewat `addEdge_3001`, koneksi dibuat di kedua arah sekaligus.

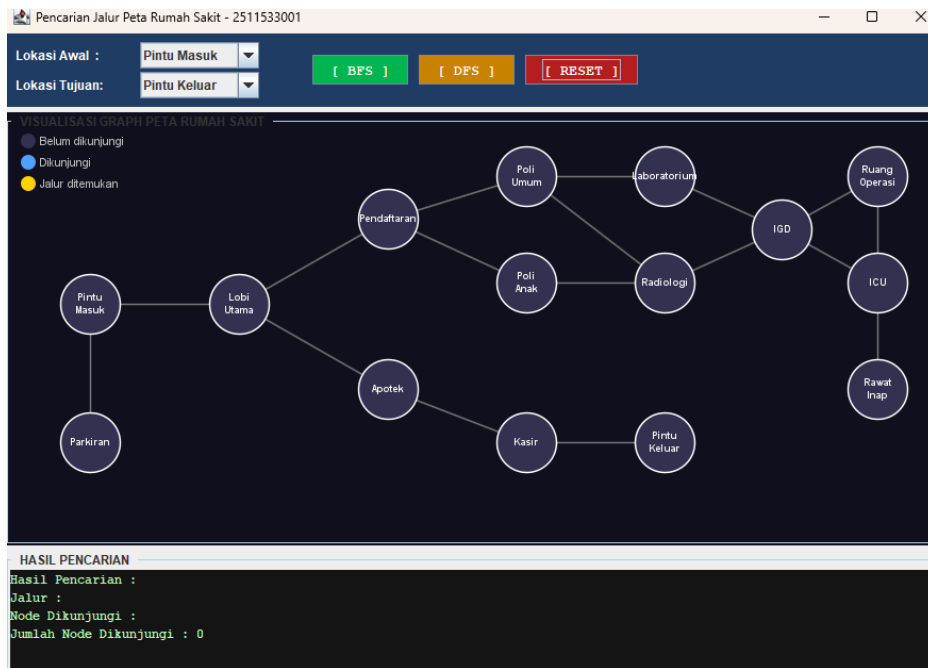
Untuk pencarian jalur, tersedia dua algoritma. `BFS_3001` menelusuri graph secara melebar menggunakan antrian, memproses semua tetangga di level yang sama sebelum lanjut ke level berikutnya sehingga jalur yang ditemukan cenderung merupakan jalur dengan jumlah simpul paling sedikit. `DFS_3001` bekerja sebaliknya, langsung mendalami satu cabang secara rekursif sampai menemukan tujuan atau mentok, baru kemudian kembali ke percabangan sebelumnya. Keduanya mencatat relasi parent tiap node selama penelusuran, yang kemudian

digunakan oleh bangunJalur_3001 untuk merekonstruksi jalur dari node tujuan balik ke node awal.

Hasil pencarian ditampilkan lewat displayPath_3001 yang menyimpan daftar node yang dikunjungi dan jalur yang ditemukan ke dalam state panel, sekaligus menampilkan ringkasannya di area teks bawah berupa jalur, urutan node yang dikunjungi, dan jumlah totalnya. Visualisasi graph ditangani oleh displayGraph_3001 yang dipanggil setiap kali panel perlu digambar ulang. Node memiliki tiga warna berbeda, gelap jika belum dikunjungi, biru jika sudah dikunjungi, dan kuning jika menjadi bagian dari jalur yang ditemukan. Edge yang menjadi bagian jalur juga ditampilkan lebih tebal berwarna kuning untuk memperjelasnya. Tombol Reset lewat resetGraph_3001 mengosongkan semua state dan memicu repaint sehingga graph kembali ke kondisi awal.

Output:





BAB III

PENUTUP

3.1 Kesimpulan

Dari praktikum mengenai implementasi struktur data pohon (Binary Tree) dan graf (Graph) pada Java, dapat disimpulkan bahwa kedua struktur tersebut merupakan representasi data non-linear yang memiliki cara kerja dan kegunaan masing-masing dalam pengelolaan serta penelusuran data.

Binary Tree yang diimplementasikan melalui class Node dan BTree memungkinkan data disimpan dalam bentuk hierarki di mana setiap simpul memiliki maksimal dua anak, yaitu kiri dan kanan. Pada BTreeDriver, pohon dibangun secara manual dengan menyambungkan sembilan simpul mulai dari root hingga level terbawah, kemudian dilakukan penelusuran menggunakan tiga metode yang berbeda. Inorder menelusuri simpul kiri terlebih dahulu, kemudian simpul saat ini, lalu simpul kanan. Preorder dimulai dari simpul saat ini sebelum menelusuri kiri dan kanan. Sedangkan Postorder menelusuri kiri dan kanan lebih dulu sebelum memproses simpul saat ini. Masing-masing metode menghasilkan urutan data yang berbeda sesuai kebutuhan pemrosesan.

Selain Binary Tree, praktikum ini juga mengimplementasikan struktur Graph melalui class GraphTraversal menggunakan Adjacency List berbasis HashMap. Graph yang dibangun bersifat tidak berarah sehingga setiap koneksi antar simpul berlaku dua arah. Penelusuran dilakukan menggunakan dua pendekatan yaitu DFS yang menelusuri graph secara mendalam dengan rekursi, dan BFS yang menelusuri graph secara melebar menggunakan antrian. Keduanya dijalankan dari titik awal yang berbeda untuk memperlihatkan perbedaan urutan kunjungan simpul yang dihasilkan meskipun graph yang digunakan sama.

Dengan demikian, praktikum ini tidak hanya memberikan pemahaman mengenai konsep dan cara kerja Binary Tree serta Graph, tetapi juga meningkatkan kemampuan dalam berpikir rekursif, memahami struktur data non-linear, serta menerapkan algoritma penelusuran yang relevan dalam pengembangan program berbasis Java.

3.2 Saran

Sebaiknya dosen menyelenggarakan sesi pra-praktikum di kelas sebagai pengantar materi, agar mahasiswa memperoleh pemahaman awal yang lebih baik. Dengan demikian, potensi kepanikan atau kesalahan selama praktikum dapat diminimalkan. Disarankan agar materi praktikum dibagikan terlebih dahulu melalui platform iLearn.

DAFTAR PUSTAKA

GeeksforGeeks. (2024). *Breadth First Search or BFS for a Graph*. Diakses dari <https://www.geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph/>

GeeksforGeeks. (2024). *Depth First Search or DFS for a Graph*. Diakses dari <https://www.geeksforgeeks.org/depth-first-search-or-dfs-for-a-graph/>